Exploring Semantic vs. Syntactic Features for Unsupervised Learning on Application Log Files

Egil Karlsen
Faculty Computer Science
Dalhousie Univeristy
Halifax, Canada
egil.karlsen@dal.ca

Rafael Copstein
Faculty Computer Science
Dalhousie University
Halifax, Canada
rafael.copstein@dal.ca

Xiao Luo
Information Technology
IUPUI
Indiana, USA
luo25@iupui.edu

Jeff Schwartzentruber
Faculty of Computer Science
Dalhousie Univeristy
Halifax, Canada
jeffrey.schwartzentruber@gmail.com

Bradley Niblett
Executive Consultant
2Keys Corporation
Ottawa, Canada
bniblett@2keys.ca

Andrew Johnston
VP Industry Relations
2Keys Corporation
Ottawa, Canada
ajohnston@2keys.ca

Malcolm I. Heywood
Faculty of Computer Science
Dalhousie Univeristy
Halifax, Canada
mheywood@cs.dal.ca

Nur Zincir-Heywood
Faculty of Computer Science
Dalhousie Univeristy
Halifax, Canada
zincir@cs.dal.ca

2023 7th Cyber Security in Networking Conference (CSNet) | 979-8-3503-4287-1/23/$31.00 ©2023 IEEE | DOI: 10.1109/CSNET59123.2023.10339765

***Abstract*—This paper explores the effect of traditional TF-IDF and state-of-the-art Large Language models towards anomaly detection in application log files. Four unsupervised learning algorithms are used on three application log files using these models. Results show that each model has advantages and disadvantages depending on the log file used.**

***Index Terms*—Application log analysis, unsupervised learning, TF-IDF features, semantic features, large language models, and BERT models.**

I. Introduction

On the Internet, service/system activity is monitored via log files, which include details of network events and the timestamps at which the events occurred. This standard pratice results in the generation of many different historical logs for each system, where log data may take many different forms. These logs are typically aggregated in a central system (e.g. SIEM) for log analysis and subsequent identification of anomalies. There exist several different approaches for log analysis, from writing regex parsers for specific log lines for analyzing specific queries to supervised machine learning based approaches for identifying malicious behaviours. Natural Language Processing (NLP) techniques have come into the spotlight as an alternative method for log analysis, due to its inherent benfits of increased scalability and generalizability for feature extraction compared to traditional approaches [5], [9].

Research in the application of NLP techniques by Boffa et al. for feature extraction in log analysis have identified the use of Word2Vec and TF-IDF as effective methods of deriving vector space representations of aggregated security data from exposed honeypots [5]. In addition, Copstein et al. explore the use of TF-IDF in combination with unsupervised machine learning algorithms on publicly available network traffic and application log data, showing it as a valid strategy for anomaly detection [6], [9]. However, exploring the efficacy of NLP based approaches is relatively recent and have not been fully evaluated. Thus, our objective is to explore the use of both TF-IDF based (syntactic) and Large Language Model (LLM) based (semantic) feature extraction techniques on three different applications logs in combination with four unsupervised machine learning algorithms. We expand on the previous work by utilizing a syntactic extraction technique proposed in [6] based on TF-IDF. In addition we extract semantic features by utilizing different LLMs on the same set of log files [7]. To the best of our knowledge, this research is the first to compare the syntactic versus semantic feature extraction using unsupervised learning on application log analysis. The research contributions include:

- Evaluating syntactic and LLM based semantic feature extraction approaches using four unsupervised learning techniques on application logs;
- Exploring which combination is a better generalizable approach to application log analysis and anomaly detection;
- Interpreting the results through data visualization

The rest of the paper is organized as follows. Section II summarizes related literature. Section III introduces the proposed approach and the methodology followed. Section IV details the evaluations, results and comparisons to the related work. Finally, conclusions are drawn and the future work is discussed in Section V.

II. Literature Review

Log analysis, a crucial area of cybersecurity, involves various log abstraction techniques for diverse types of log data analysis. The abstraction techniques often produce output that is used as a feature set for intrusion detection or anomaly detection in data analysis. Nguyen et al. proposed a supervised ensemble learning system for the classification of abnormal security events by combining outputs from multiple supervised machine learning models [22]. This system was tested on the ECML/PKDD 2007 and HTTP CSIC 2010 datasets, proving

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on May 30,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

the importance of adapting intrusion detection systems in diverse network environments.

Bhatnagar et al. compared the performance of four supervised machine learning algorithms for intrusion detection [4]. They extracted and learned features from raw log data using a deep belief network on both the ECML/PKDD 2007 and HTTP CSIC 2010 datasets. The resultant hybrid feature set was used to train four different supervised classifiers for intrusion detection. Vartouni et al. also presented a feature extraction method using deep belief networks and parallel feature fusion techniques [20]. They evaluated the benefits of dimensionality reduction, synthetic feature fusion, and baseline n-gram representation by training unsupervised anomaly detection algorithms and comparing their outlier detection performance using accuracy and F1-score. Copstein et al. [6] explored syntactic features based on the TF-IDF technique for intrusion detection in application log files. They showed that k-menas clustering using a syntactical approach distinguishes malicious behavior very effectively. Moreover, recent studies have looked into the use of BERT for log analysis in VM failure prediction. Nam et al. proposed a model that used BERT to extract the sentence embedding representation of each log, which was then fed into a pre-trained convolutional neural network to predict failure [21]. Ott et al. evaluated the use of language models BERT, GPT-2, and XL-Net for anomaly detection in cloud system log data [23]. Lastly, Zhang et al. modified the BERT base model to perform weighted part-of-speech tagging on log templates produced by Drain to improve classification for log events across different system logs [27]. Table I present the overview of the state-of-the-art.

TABLE I
LITERATURE OVERVIEW

Paper	Unsupervised	Syntactic	Semantic	Application Log
[22]	NO	NO	NO	YES
[4]	NO	YES	NO	YES
[20]	YES	YES	NO	YES
[6]	YES	YES	NO	YES
[13]	YES	NO	NO	YES
[15]	YES	NO	NO	YES
[21]	NO	NO	YES	NO
[23]	NO	NO	YES	NO
[27]	NO	NO	YES	NO
This Study	YES	YES	YES	YES

III. METHODOLOGY

This section explores the different datasets, feature extraction methods, visualization methods, and unsupervised machine learning algorithms utilized in this research.

A. Datasets

Since one of the objectives of this research is to explore a generalizable approach to application log analysis for security purposes, it is important to employ data from different sources. Therefore, three publicly available application logs are employed in this research. The characteristics of the logs are summarized in Table II and Table III. The details of each dataset is described as follows.

- **ECML/PKDD Dataset:** Generated for the Discovery Challenge 2007, this log contains 50,115 anonymized log records representing HTTP packet information and includes both normal and attack behaviours [8]. It has the highest number of words among all datasets used in this work and is hereafter referred to as PKDD.
- **CSIC Dataset:** Generated in 2010 by the Information Security Institute of the Spanish Research National Council, this log provides an updated alternative to the KDD cup dataset for intrusion detection research on web application logs [10]. Produced by simulating an e-commerce site, it encompasses several types of simulated attacks and is hereafter referred to as CSIC.
- **Apache Access Dataset:** Generated by Indramayu State Polytechnic University in 2022, this log comprises 37,693 log records from web server access logs and represents a regular application log file [12]. It has the lowest number of words among all datasets used in this work and is hereafter referred to as AA.

TABLE II
SUMMARY OF THE DATASETS

	PKDD		CSIC		AA	
Class	#Lines	%	#Lines	%	#Lines	%
Normal	35006	69.85%	36,000	59.01%	29,778	85.71%
Anomaly	15109	30.15%	25,000	30.99%	4,965	14.29%
Total	50,115	100%	61,000	100%	34,743	100%

TABLE III
NLP CHARACTERISTICS OF THE DATASETS

Dataset	Word Length	Word Count	Character Count
PKDD	11.49	92.1	1124.25
Standard Deviation	1.67	12.32	168.01
CSIC	15.94	27.55	467.02
Standard Deviation	3.50	0.90	104.13
AA	12.22	16.85	222.13
Standard Deviation	2.87	1.83	46.07

B. Feature Extraction

This study examines two feature extraction techniques (syntactic and semantic) in NLP, applied with no a priori information on the data's log formats. The semantic method focuses on extracting meaning from contextual relationships within the log, while the syntactic approach identifies key words through their frequency in the log file's syntax representation.

1) Syntactic: TF-IDF based Feature Extraction: TF-IDF is based on the understanding that a term's TF-IDF score increases as its frequency in a given document increases and its frequency in the entire document model decreases. High TF-IDF scores, therefore, indicate outliers. In this work, we use a lightweight syntactical feature extraction technique, TF-IDF for application log analysis following the work in [1]. In this case, for a log line d with tokens ts, and a log set D of size N, TF-IDF vector representation can be calculated using Eqs. 1, 2, and 3 [1].

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on May 30,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

$$tf(t,d) = freq(t,d) \tag{1}$$

$$idf(t,D) = log(\frac{N+1}{count(k \in D : t \in k)+1}) + 1 \tag{2}$$

$$tfidf(t,d,D) = tf(t,d) \times idf(t,D) \tag{3}$$

Given the verbosity and variability of TF-IDF representations in logs, three TF-IDF features, namely average, minimum, and maximum values are extracted from each log line for a concise syntactic feature set. Additionally, four more syntactic features are calculated from each log line: Alphanumeric Ratio, Average Character (numeric ASCII representation), Character Count, and Word Count. Thus in total seven syntactic features are extracted using this technique per log line.



Fig. 1. An example of TF-IDF calculations on a log file

2) Semantic: LLM based Feature Extraction: In this research, we use the state-of-the-art BERT transformer model for the semantic approach to derive embedding representations from each log line [25]. The Bidirectional Encoder Representation from Transformers model (BERT) is another tool we use for the semantic approach to get the embedding representation of each log line [7]. BERT, which leverages the transformer architecture, was previously used on security log data by Guo et al. [11] to identify anomalous events in log information.

To perform sentence or sequence-level analysis using BERT variants, we extract the hidden states or token embeddings from the model's final layer and apply mean pooling. This process compresses the token embeddings into a fixed-length sequence representing the meaning of the input sequence [24]. We added a mean pooling layer after the final layer of each model to extract the sentence embedding from the language models used in this study (see Figure 2).

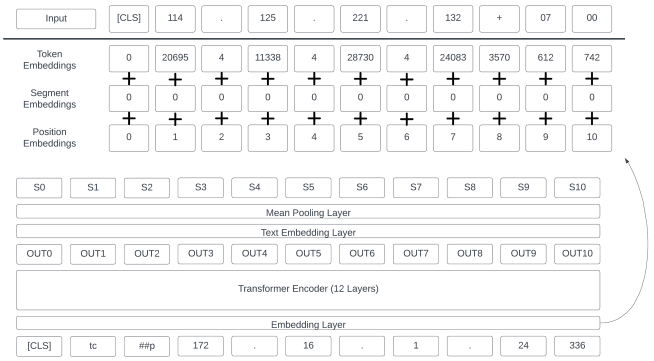


Fig. 2. An example of BERT Embedding on a log file

For the purposes of this research we utilize the distilRoBERTa model which is a distilled varient of the BERT model [25]. A distilled model leverages knowledge distillation for improved inference time and reduced model complexity, the distillation process involves using large language models to train smaller ones through a teacher-student compression technique, aiming for knowledge transfer and distillation. RoBERTa, the teacher model, outperformed similarly sized BERT-style models in the GLUE benchmark in the study by Liu et al. [17] .The choice of the distilled model was due to its faster speed and lower computational overhead, offering double the inference time for DistilRoBERTa.

Each log record is input into the DistilRoBERTa model, producing 768 hidden states that form the sentence embedding information (Figure 2). These embeddings serve as the semantic feature set representing a log record. The DistilRoBERTa model underwent no fine-tuning or modification, as our goal is to determine the potential of semantic and syntactic feature extraction strategies as out-of-the-box methods with default parameters on different application log files.

C. Unsupervised Learning Techniques

The unsupervised learning techniques used in this work include Self Organizing Maps (SOM), K-means, Isolation Forest (iForest), and Agglomerative Hierarchical (Agglomerative) clustering. They are chosen based on the related work discussed in Section II. These unsupervised machine learning algorithms are typically used for anomaly detection in security logs. Further details regarding unsupervised learning (clustering) algorithms can be found in [2].

The K-Means algorithm is a well-established iterative clustering algorithm [3] [18]. This algorithm works iteratively by assigning data points to k clusters and (re)computing the cluster centroid via euclidean distance. The value k is a

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on May 30,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

parameter provided to the algorithm and defines the number of clusters.

The Agglomerative clustering is a bottom-up hierarchical clustering algorithm [28]. It starts with N clusters each with one instance and merges two closest clusters at each step. This process iterates until a single unified cluster exists.

The Isolation Forest algorithm focuses on outlier/anomaly isolation rather than normal instance modeling [16]. The algorithm isolates anomalies using binary trees. The algorithm iterates until either each data point has been separated or the provided maximum tree depth has been reached. Upon convergence of the algorithm, the 'anomaly score' for a data point can be computed.

The Self Organizing Map (SOM) is an artificial neural network based unsupervised learning algorithm [14]. SOM produces a two-dimensional representation of high dimensional data which preserves the pairwise distance relationship of the original data. The goal of SOM learning algorithm is to cause different parts of the neural network to respond to the certain input mappings. During mapping, there is a single winning neuron whose weight vector lies closest to the input vector, e.g. Euclidian distance.

D. Analysis and Visualization

We use two visualization algorithms, SOM and t-SNE, to discover the resulting clusters or distributions of the data for syntactic and semantic feature sets, respectively. The t-SNE visualization is a non-linear dimensionality reduction technique which attempts to preserve small pairwise distances [26]. The SOM visualization produces a neuron map to visualize a high dimensional dataset on a two-dimensional plane [14]. Both visualizations are shown in Figure 7.

IV. Evaluation

A. Evaluation Structure

In exploring anomaly detection with both the syntactic and semantic approach to log data feature extraction, we conduct a benchmark using various unsupervised learning techniques. This involves comparisons of the syntactic and semantic feature extraction approaches over three distinct datasets (PKDD, CSIC, and AA) using four unsupervised machine learning algorithms, namely K-Means, Agglomerative, Isolation Forest, and Self Organizing Maps.

It should be noted here that during evalutions, each dataset was split into 75% training and 25% testing, using stratified sampling to maintain the original class distributions of the data. For the purposes of this research the training and test data were reduced to a binary representation of their original classes, that is to say classes associated with malicious behavior were all labelled as ANOMALY and the remanining were labeled as NORMAL. Given that unsupervised learning algorithms are used towards anomaly detection, no labels are used during training (even though the log files include ground truth, i.e. labels). Thus, labels are introduced after training to be able to measure and compare the performances of different approaches for testing. Thus, the resulting clusters of K-Means, Agglomerative and SOM algorithms were labelled based on majority vote post-training. While the Isolation Forest utilized anomaly-labeled data for the identification of abnormalities.

TABLE IV
HyperParameter Setting

Algorithm	PKDD	AA	CSIC
SOM	Grid = 40*40, Epochs = 10000		
K-Means	k=100	k=100	k=100
Agglomerative	c=10	c=100	c=50
Isolation Forest	i=0.3	i=0.15	i=0.3

For the evaluation of the models, we use three performance metrics: Weighted F1-score, Precision, and Recall. They are calculated from confusion matrices derived from the training and testing of each technique. Precision (P), the proportion of relevant true positives, is given by $P = \frac{TP}{TP+FP}$. Recall (R) represents the ratio of true positives relative to all positives in the data, computed as $R = \frac{TP}{TP+FN}$. The F1-score, which measures model performance, is determined from Precision and Recall through the formula $F1 = \frac{2PR}{P+R}$.

In this work, we utilize the Weighted F1-Score, which calculates the per class weighted mean of all F1-scores considering the class proportion, S, in a given dataset, with the equation being $F1_{\text{weighted}} = \sum \frac{F1_{\text{class}}}{S_{\text{class}}}$. The algorithm rendering the highest performance was ascertained based on the Weighted F1-scores obtained from the evaluations [19].

Apart from evaluating the performance of unsupervised learning algorithms using syntactic and semantic feature extraction, computational cost, in terms of time required, for feature extraction and learning model building is also measured. Computational cost is considered a crucial metric due to the high volume, velocity, and veracity of log data, necessitating fast identification of anomalies, particularly in the case of real-time analytics.

B. HyperParameters for Learning Algorithms Used

The following section outlines the process of how the hyperparameters are tuned on each model for each dataset. Table IV shows the hyperparameters that are chosen.

The optimal parameters for each of the unsupervised learning models used in this work are derived by empirically. The value of k for k-means clustering was determined by comparing the training classification performance across models with an increasing number of clusters. The elbow study depicted in Figures 3, 5, and 4 shows the results of the elbow study for each algorithm on each log, repsectively. A power law curve is utilized to approximate the change in the weighted F1-score as k increases, revealing a plateau between 90 and 110 clusters across different log files. As a result, k value of 100 is selected for all k-means experiments, as shown in Table IV. For Agglomerative clustering, the optimal value of c is derived by evaluating model performance with increasing values of c, using the weighted F1-score as a measure, with the best performing values listed in Table IV. For Isolation Forest, the primary tunable parameter is the contamination ratio, which

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on May 30,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

is calculated based on the proportion of ANOMALY labeled data points in each log file. Finally, for the Self-Organizing Maps, various parameters are explored empirically, including different map/grid sizes across all log files, and both syntactic and semantic feature sets. The resulting hyperparameter settings for all learning algorithms used are presented in Table IV.

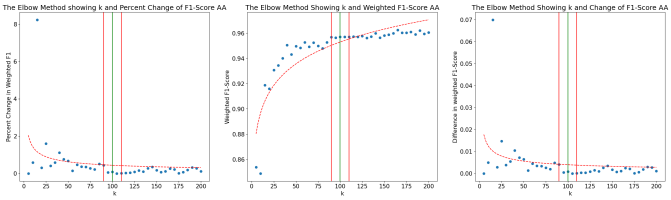


Fig. 3. Elbow study of K-Means on Apache Access log

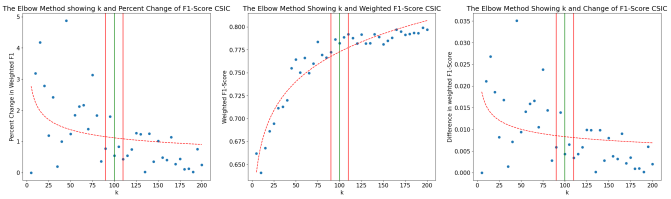


Fig. 4. Elbow study of K-Means on CSIC log

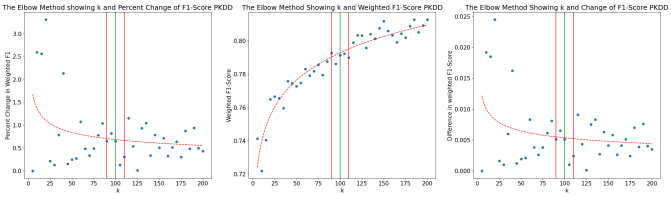


Fig. 5. Elbow study of K-Means on PKDD log

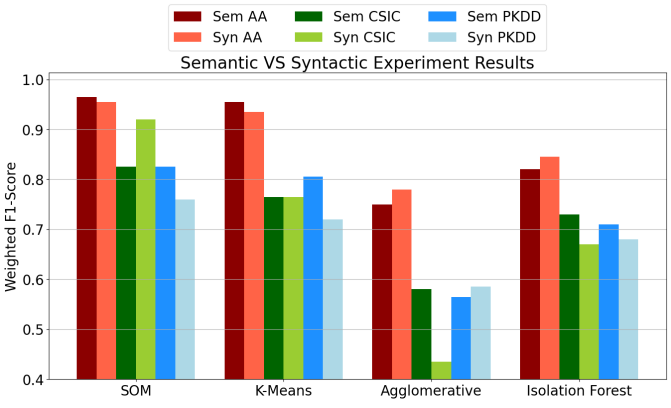


Fig. 6. Semantic (Sem) vs. Syntactic (Syn) feature extraction evaluation results: Weighted F1-Score

C. Results

Figure 6 presents the weighted F1-score evaluation results for the four unsupervised learning techniques used with semantic and syntactic feature sets across three log datasets.

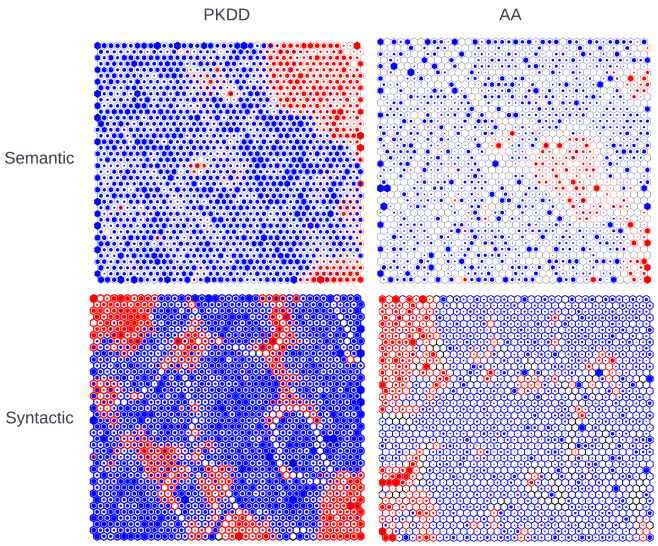


Fig. 7. SOM Hit Map Visualizations of PKDD and AA datasets

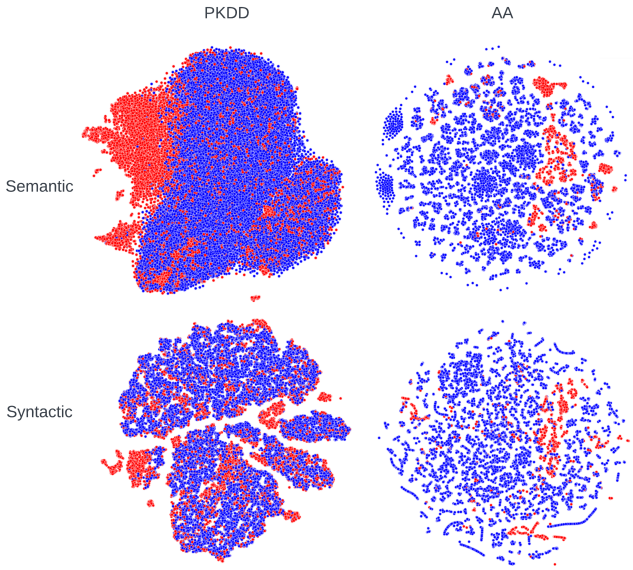


Fig. 8. t-SNE Visualizations of PKDD and AA datasets

Based on the weighted F1-score results, The best performing unsupervised learning algorithms are SOM and K-means. On the AA log file, semantic feature extraction yileds to a better performance by around 2% compared to syntactic approach. On the PKDD log file, semantic feature extraction techniques results in an increase of 6% in SOM and 8% in K-Means over syntactic approach. However, on the CSIC log file, the syntactic approach results in an increase of around 10% when using SOM compared to the semantic approach. This is the largest difference in performances between the two techniques across all results. Moreover, semantic and syntactic approaches acheive the same result on the CSIC log using the K-means

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on May 30,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPUTATIONAL COST (IN SECONDS): SEMANTIC VS SYNTACTIC APPROACHES

	Feature Extraction Time & Model Training Time					
	AA		CSIC		PKDD	
Algorithm	Syntactic	Semantic	Syntactic	Semantic	Syntactic	Semantic
Feature Extraction	5	44	11	138	27	217
Isolation Forest Model	19	58	44	73	30	73
Agglomerative Model	25	197	60	471	54	429
K-Means Model	3	35	5	84	5	102
SOM Model	346	1,300	480	2,068	531	1,906

algorithm.

On the other hand, Isolation forest and agglomerative clustering algorithms perform modestly across all datasets compared to the K-Means and SOM models. Given that these models offer no additional benefits such as visualization through SOM hitmaps, they can be considered less effective for anomaly detection using these feature extraction techniques.

Insights from the t-SNE and SOM hit map visualization methods seen in Figures 8 and 7 for all datasets align with these results. Semantic feature sets consistently demonstrate better separation between normal and anomalous data in the visualizations.

The PKDD dataset, characterized by its anonymization, presents a challenge in separating in terms of normal and anomalous behavior. Although both t-SNE and SOM hit maps display a mix of data types for both syntactic and semantic feature sets, a unique cluster of entirely anomalous data is evident in the semantic t-SNE plot, and some anomalously labeled neurons appear in the top left of the semantic SOM hit map. This suggests that information loss during anonymization may have contributed to these results. Whereas the Apache Access visualizations clearly align with the learning model performance results, with distinguishable clusters seen in both the syntactic and semantic t-SNE plots and SOM hit maps, reinforcing its high performance in both syntactic and semantic learning models with the weighted F1-scores above 90%.

D. Discussion

This research has explored four different unsupervised learning techniques, employing two NLP inspired feature extraction methods on three distinct application logs. On the AA log, all learning algorithms demonstrate high weighted F1-scores with both of the feature extraction techniques, although semantic feature extraction has a slight edge. The SOM and K-Means algorithms have the best performances on this data.

The trend continues with the PKDD log, where semantic feature extraction outperforms syntactic approach, particularly with SOM and K-means algorithms. The lower performances achieved while using the syntactic approach are likely due to the anonymized nature of the dataset, making it more challenging to differentiate anomalous behavior from normal behaviour.

However, all algorithms seem to prefer the syntactic feature extraction on the CSIC log as demonstrated by their higher performance compared to the semantic approach. This seems to indicate the potential of syntactic feature extraction in simple applicatuion datastes to disambiguate anomolous behavior.

Another important aspect to discuss is the computational cost (time) associated with the feature extraction and model training processes. As presented in Table V, the semantic feature extraction technique, although producing better results in two out of three log files, require significantly more time and computational resources. Conversely, the syntactic feature extraction allows for a much faster execution.

In summary, assessing both computational costs and performances, K-Means emerges as the top performer with low cost, while SOM algorithms innately offer valuable visualization and interpretability benefits but at the expense of significant cost. Among the two feature extraction techniques, semantic comes out on top. Semantic extraction yields better results in two out of three datasets, and matching performance in the third, albeit at the cost of higher computational costs.

V. CONCLUSION AND FUTURE WORK

This research examined the effectiveness of semantic and syntactic feature extraction methodologies, coupled with unsupervised machine learning algorithms for anomaly detection, across three distinct application log sets. To the best of our knowledge this study is the first to evaluate LLM-based semantic feature extraction versus TF-IDF based syntactic feature extraction in application specific log analysis.

In two of the three datasets, the LLM-based semantic feature extraction demonstrated superior performance compared to the syntactic approach, indicating its potential promise. Across all log files, both K-means and SOM unsupervised learning models acheived high performance. However K-Means has significantly low cost in both training and inference phases.

Furthermore, our investigation highlighted some limitations of large language models in log analysis, particularly their high computational cost. Future research will aim to reduce this computational cost and refine the sentence embedding representations of log files through the exploration and adaptation of diverse large language models.

ACKNOWLEDGEMENT

This research is supported by an NSERC Alliance Grant. The research is conducted as part of the Dalhousie NIMS Lab[1].

[1]https://projects.cs.dal.ca/projectx/

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on May 30,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Akiko Aizawa. An information theoretic perspective of tf-idf measures. *Information Processing Management*, 39:45–65, 01 2003.

[2] E. Alpaydin. *Introduction to Machine Learning, second edition*. Adaptive Computation and Machine Learning series. MIT Press, 2009.

[3] G. H. Ball and David J. Hall. Isodata, a novel method of data analysis and pattern classification. 1965.

[4] Mansi Bhatnagar, Gregor Rozinaj, and Puneet Kumar Yadav. Web intrusion classification system using machine learning approaches. In *2022 International Symposium ELMAR*, pages 57–60, 2022.

[5] Matteo Boffa, Giulia Milan, Luca Vassio, Idilio Drago, Marco Mellia, and Zied Ben Houidi. Towards nlp-based processing of honeypot logs. pages 314–321, 2022.

[6] Rafael Copstein, Egil Karlsen, Jeff Schwartzentruber, Nur Zincir-Heywood, and Malcolm Heywood. Exploring syntactical features for anomaly detection in application logs. *it - Information Technology*, 64(1-2):15–27, 2022.

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2018.

[8] ECML/PKDD. Ecml/pkdd 2007 discovery challenge, September 2021. https://gitlab.fing.edu.uy/gsi/web-application-attacks-datasets/-/tree/master/ecml_pkdd.

[9] Brian Gallagher and Tina Eliassi-Rad. Classification of http attacks: A study on the ecml/pkdd 2007 discovery challenge. 01 2009.

[10] Carmen T Giménez, Alejandro P Villegas, and Gonzalo Á. Marañón. HTTP Dataset CSIC 2010, 2010.

[11] Haixuan Guo, Shuhan Yuan, and Xintao Wu. Logbert: Log anomaly detection via bert. pages 1–8, 2021.

[12] Muhammad Anis Al Hilmi, Kurnia Adi Cahyanto, and Muhamad Mustamiin. Apache web server - access log pre-processing for web intrusion detection, 2020.

[13] Gunes Kayacık, A. Zincir-Heywood, and Malcolm Heywood. A hierarchical som-based intrusion detection system. *Engineering Applications of Artificial Intelligence*, 20:439–451, 06 2007.

[14] Teuvo Kohonen. The self-organizing map. *Neurocomputing*, 21(1):1–6, 1998.

[15] Md Tahmid Rahman Laskar, Jimmy Xiangji Huang, Vladan Smetana, Chris Stewart, Kees Pouw, Aijun An, Stephen Chan, and Lei Liu. Extending isolation forest for anomaly detection in big data via k-means. *ACM Trans. Cyber-Phys. Syst.*, 5(4), sep 2021.

[16] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. pages 413–422, 2008.

[17] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019.

[18] S. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[19] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. The MIT Press, 2012.

[20] Ali Moradi Vartouni, Mohammad Teshnehlab, and Saeed Sedighian Kashi. Leveraging deep neural networks for anomaly-based web application firewall. *IET Information Security*, 13(4):352–361, 2019.

[21] Sukhyun Nam, Jae-Hyoung Yoo, and James Won-Ki Hong. Vm failure prediction with log analysis using bert-cnn model. pages 331–337, 2022.

[22] Hai Thanh Nguyen and Katrin Franke. Adaptive intrusion detection system via online machine learning. In *2012 12th International Conference on Hybrid Intelligent Systems (HIS)*, pages 271–277, 2012.

[23] Harold Ott, Jasmin Bogatinovski, Alexander Acker, Sasho Nedelkoski, and Odej Kao. Robust and transferable anomaly detection in log data using pre-trained language models. *CoRR*, abs/2102.11570, 2021.

[24] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *CoRR*, abs/1908.10084, 2019.

[25] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *ArXiv*, abs/1910.01108, 2019.

[26] Laurens van der Maaten and Geoffrey Hinton. Viualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 11 2008.

[27] Jing Zhang, Zezhou Li, Xianbo Zhang, Feng Lin, Chao Wang, and Xingye Cai. Posbert: Log classification via modified bert based on part-of-speech weight. pages 979–983, 2022.

[28] Ying Zhao, George Karypis, and Usama Fayyad. Hierarchical clustering algorithms for document datasets. *Data Min. Knowl. Discov.*, 10:141–168, 03 2005.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on May 30,2026 at 08:01:46 UTC from IEEE Xplore. Restrictions apply.